第二阶段：vdot.vv RTL 实现说明

工大喵喵队 2026 年 11 月

一、改动文件

vdot 是在 VIMacU 里加一条旁路数据通路，复用原来的发射口、ROB 标签、写回端口和异常通道。改 4 个已有文件，新增 1 个文件：

文件	类型	作用
`decode/VecDecoder.scala`	修改	把 custom-0 指令字识别成 vdot
`decode/DecodeUnit.scala`	修改	译码阶段检查 vtype/vstart 非法
`decode/isa/bitfield/RiscvInst.scala`	修改	让自定义 opcode 进入向量算术路径
`fu/wrapper/VIMacU.scala`	修改	实例化数据通路，做输出/异常/vxsat 选择
`fu/wrapper/Vdot8Datapath.scala`	新增	8 路乘加树、两级寄存器

完整补丁在 `rtl/xiangshan/xiangshan-vdot-complete.patch`，大小 6,317 字节。归档目录里的 `xiangshan.patch` 只有 `git diff` 对已跟踪文件的输出，不含新增的 `Vdot8Datapath.scala`，提交时要用完整补丁，或者把 Scala 文件单独带上。

二、关键代码

Vdot8Datapath.scala · rtl/xiangshan

```
14 class Vdot8Datapath extends Module {
15   val io = IO(new Bundle {
16     val vs1 = Input(UInt(128.W))
17     val vs2 = Input(UInt(128.W))
18     val enable1 = Input(Bool())
19     val enable2 = Input(Bool())
20     val vd = Output(UInt(128.W))
21   })
22   val products = (0 until 8).map { i =>
23     RegEnable(io.vs1(8*i+7,8*i).asSInt * io.vs2(8*i+7,8*i).asSInt, io.enable1)
24   }
25   val pairs = products.grouped(2).map(x => x(0) +& x(1)).toSeq
26   val halves = pairs.grouped(2).map(x => x(0) +& x(1)).toSeq
27   val sum = halves(0) +& halves(1)
28   val result = RegEnable(sum.pad(32).asUInt, io.enable2)
29   io.vd := Cat(0.U(96.W), result)
```

图 1 Vdot8Datapath 关键代码

VdotSpec 和译码：

```
object VdotSpec {
  val instruction = BitPat("b0000001??????????010?????0001011")
  val fuOp = BitPat("b100000000")
  def isDot(op: UInt): Bool = op === "h100".U
}
```

VecDecoder 里加一行：

```
xiangshan.backend.fu.wrapper.VdotSpec.instruction ->
  OPMVV(T, FuType.vimac, xiangshan.backend.fu.wrapper.VdotSpec.fuOp,
        F, T, F, UopSplitType.VEC_VVV)
```

DecodeUnit 里加非法配置判断：

```
isAmocasQIllegal ||
(xiangshan.backend.fu.wrapper.VdotSpec.instruction === inst.ALL &&
  (io.enq.vtype.illegal || io.enq.vtype.vsew =/= 0.U ||
   io.enq.vtype.vlmul =/= 0.U || io.enq.vstart =/= 0.U))
```

RiscvInst.isVecArith 放宽：

```
this.OPCODE5Bit === OPCODE5Bit.OP_V ||
  xiangshan.backend.fu.wrapper.VdotSpec.instruction === this.ALL
```

数据通路：

```
class Vdot8Datapath extends Module {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val enable1 = Input(Bool())
    val enable2 = Input(Bool())
    val vd = Output(UInt(128.W))
  })
  val products = (0 until 8).map { i =>
    RegEnable(io.vs1(8*i+7,8*i).asSInt * io.vs2(8*i+7,8*i).asSInt, io.enable1)
  }
  val pairs  = products.grouped(2).map(x => x(0) +& x(1)).toSeq
  val halves = pairs.grouped(2).map(x => x(0) +& x(1)).toSeq
  val sum    = halves(0) +& halves(1)
  val result = RegEnable(sum.pad(32).asUInt, io.enable2)
  io.vd := Cat(0.U(96.W), result)
}
```

`asSInt` 把字节当有符号数，`0x80` 是 −128，`0xFF` 是 −1；`+&`是带位宽扩展的加法，避免截断；`RegEnable` 用 `enable1`/`enable2` 控制两级寄存器；`sum.pad(32)` 做符号扩展，`Cat(0.U(96.W), result)`拼成 128 位。

VIMacU 里接：

```
private val isDot = VdotSpec.isDot(fuOpType)
private val outIsDot = VdotSpec.isDot(outCtrl.fuOpType)
private val dot = Module(new Vdot8Datapath)
require(cfg.destDataBits == 128 && latency == 2)
dot.io.vs1 := vs1
dot.io.vs2 := vs2
dot.io.enable1 := regEnable(1) && isDot
dot.io.enable2 := regEnable(2)

mod.io.fire := io.in.valid && !isDot
...
io.out.bits.res.data := Mux(outIsDot, dot.io.vd, mgu.io.out.vd)
io.out.bits.res.vxsat.get := !outIsDot && (outVxsat & mgu.io.out.active).orR
```

`mod.io.fire` 加`!isDot`，让原来的 MGU/掩码路径不处理 vdot；`Mux` 只在 vdot 时选点积结果；`vxsat` 对 vdot 清零。

三、位宽设计

量	范围
单个乘积	−16,256 ～ 16,384
8 项和	−130,048 ～ 131,072
19 位有符号	−262,144 ～ 262,143

8×16,384 = 131,072，18 位有符号最大 131,071，会溢出，所以总和用 19 位。32 位输出足够。

四、编译和运行

在香山根目录：

```
cd $NOOP_HOME
git apply /path/to/rtl/xiangshan/xiangshan-vdot-complete.patch
make emu CONFIG=KunminghuV2MinimalConfig JVM_XMX=12G EMU_THREADS=2 -j2
```

NEMU：

```
cd $NEMU_HOME
git apply /path/to/nemu/nemu-vdot-clean.patch
make riscv64-xs-ref_defconfig
make -j$(nproc)
```

测试程序：

```
cd software
make all exception-check
```

运行：

```
cd $NOOP_HOME
./build/emu -i $HOME/risv/project/software/build/vdot-a0.bin \
  --diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
  -F $HOME/risv/artifacts/boot-minimal.bin
```

五、验证结果

RTL 分两层验证：

1.独立 SystemVerilog 原型 `rtl/standalone/vdot8.sv`，用 Icarus Verilog 跑 69,656 个向量，包含 65,536 个全字节对穷举，结果 `PASS vectors=69656 checked=69657 cancelled=4 cycles=108440`； 2.集成到香山后，用 a0 版 `vdot-a0.bin` 跑 1,030 个程序级用例 × 5 种实现变体，`HIT GOOD TRAP`、a0=0，841,260 条指令、790,553 周期，无 difftest mismatch。

独立 SV 原型和 Scala 集成版结构一致，但不是同一份代码：SV 原型做快速穷举和背压/flush 测试，Scala 集成版跑整机 difftest。

RTL 的 mtval 也分两段：DecodeUnit 在译码阶段发现 SEW/LMUL/vstart 非法，TrapInstMod 把指令字 `0x0220a18b` 填进 mtval；VL 不匹配只在 VIMacU 执行阶段发现，mtval=0。这个差异在 `05` 里有 8 种配置的实测表。

六、限制

· 只实现了 VL=8、SEW=8、LMUL=1；

· 异常覆盖 M 模式 8 种非法配置；S/VS 委托、RNMI、NMIE=0 早退路径还没跑；

· 没有做 SEW=16/32 和 VLEN=256 的实际整核版本，只讨论了扩展方式；

· 参数化原型只到独立模块级（n=4/8/16），未接进香山；没有综合、STA，所以没有面积、频率、时序数据。